

Helix VPN — Phase 2 (MVP2) Comprehensive Client Apps Plan

Helix VPN — Phase 2 (MVP2) Comprehensive Client Apps Plan

Revision: 2 Last modified: 2026-07-04T12:00:00Z

Revision 2 note (2026-07-04): this is the original research/planning brief that kicked off MVP2 documentation generation; it is preserved as a historical record and not rewritten. Two reconciliation facts for anyone reading it after the fact: (1) Stage 3 originally scoped **16** standalone documents; the delivered set consolidated to **10** (MVP2_SECURITY.md + MVP2_PERFORMANCE.md merged into MVP2_SECURITY_PERFORMANCE.md; MVP2_TECHNOLOGY_STACK.md, MVP2_BUILD_SYSTEM.md, MVP2_TESTING_QA.md, MVP2_API_REFERENCE.md, and MVP2_DEVELOPMENT_GUIDE.md content was folded into MVP2_ARCHITECTURE.md, MVP2_SHARED_CORE.md, and MVP2_SECURITY_PERFORMANCE.md) — see MVP2_OVERVIEW.md §8.1 for the as-delivered registry. (2) A 2026-07-04 gap-analysis + hardening pass added Enterprise Hardening / Production Readiness sections and Mermaid diagrams to every delivered document; that work is out of this plan's original scope and is tracked via each document's own Revision 2 changelog instead of by editing this historical plan further.

Objective

Create comprehensive Phase 2 documentation for Helix VPN client applications covering all operating systems (macOS, Windows, Linux, Android, iOS, Harmony OS, Aurora OS) and all platforms (Web, Desktop, Mobile) with maximal codebase reusability, performance, and minimal footprint.

Target Output

A documentation package matching MVP level of detail, compressed into downloadable .zip and .tar.gz archives.

Stage 1 — Deep Technology Research (Parallel)

Load: deep-research-swarm (Route A — Wide Search)

Sub-tasks:

1. **Research Cross-Platform Frameworks** — Evaluate all cross-platform technologies (Tauri, Flutter, React Native, Kotlin Multiplatform, .NET MAUI, Electron, NativeScript, Ionic, Capacitor, Qt, Avalonia, Compose Multiplatform, UniApp, Cordova, etc.) across: code reusability, performance, bundle size, memory footprint, startup time, platform coverage, maturity, ecosystem, security model, VPN-specific capabilities (raw sockets, TUN interface access, background services, kill switch).
2. **Research OS-Specific Requirements** — Deep dive into each OS platform specifics: macOS (NetworkExtension, Swift/SwiftUI), Windows (WFP, WinTUN, UWP vs Win32), Linux (NetworkManager, systemd, TUN/TAP, WireGuard), Android (VpnService, WorkManager, foreground services), iOS (NEPacketTunnelProvider), Harmony OS (HarmonyOS APIs, ArkTS/ArkUI), Aurora OS (Sailfish OS heritage, Qt-based).
3. **Research VPN-Specific Technical Requirements** — Protocol implementations (WireGuard, OpenVPN, IKEv2, Shadowsocks, VLESS, Trojan, custom protocols), encryption libraries, key management, certificate handling, split tunneling, kill switch, obfuscation, multi-hop, DNS management, IPv6 handling, MTU optimization.

4. **Research Web Platform Capabilities** — Web-based VPN clients, WebRTC-based solutions, browser extensions (Chrome, Firefox, Safari, Edge), WebSocket tunneling, WebTransport, progressive web apps (PWA) limitations for VPN.
5. **Research Shared Core / Rust Approach** — Rust as shared core library compiled to all platforms, FFI bindings, UniFFI, wasm-bindgen, performance characteristics, security benefits, library size analysis.

Output: 5 comprehensive research briefs

Stage 2 — Architecture & Technology Decision Document

Synthesize Stage 1 findings into definitive technology stack decisions.

Sub-tasks:

1. **Final Technology Stack Selection** — Recommended stack per platform with justification
2. **Shared Core Design** — Rust core library architecture
3. **Code Reusability Matrix** — Which code lives where, reuse percentages per platform
4. **Performance & Footprint Budgets** — Target metrics for each platform

**Output: architecture_decisions.md,
technology_stack.md**

Stage 3 — Comprehensive Documentation Creation (Parallel)

Load: report-writing

Sub-tasks:

1. **MVP2_OVERVIEW.md** — Executive summary, goals, scope, success criteria
2. **MVP2_ARCHITECTURE.md** — System architecture, component diagrams, data flow

3. **MVP2_TECHNOLOGY_STACK.md** — Detailed technology selections with comparisons
4. **MVP2_DESKTOP_APPS.md** — macOS, Windows, Linux client specifications
5. **MVP2_MOBILE_APPS.md** — Android, iOS, Harmony OS client specifications
6. **MVP2_WEB_CLIENT.md** — Web app, browser extensions, PWA
7. **MVP2_AURORA_CLIENT.md** — Aurora OS specific implementation
8. **MVP2_SHARED_CORE.md** — Rust core library design, API, FFI bindings
9. **MVP2_SECURITY.md** — Security architecture, threat model, cryptographic design
10. **MVP2_PERFORMANCE.md** — Performance targets, benchmarks, optimization strategies
11. **MVP2_UI_UX_SPEC.md** — Design system, component library, screen specifications
12. **MVP2_BUILD_SYSTEM.md** — CI/CD, cross-compilation, release pipeline
13. **MVP2_TESTING_QA.md** — Testing strategy, test matrix, automated testing
14. **MVP2_API_REFERENCE.md** — APIs between core and UI layers
15. **MVP2_IMPLEMENTATION_ROADMAP.md** — Phased implementation plan, milestones
16. **MVP2_DEVELOPMENT_GUIDE.md** — Developer onboarding, coding standards, contribution guide

Output: 16 comprehensive markdown documents

Stage 4 — Diagrams & Visual Assets

1. System architecture diagram
2. Component interaction diagrams
3. Platform coverage matrix diagram
4. Build pipeline diagram
5. Data flow diagrams

Output: PNG diagram files

Stage 5 — Packaging & Delivery

1. Create organized directory structure
2. Generate table of contents / index

3. Create .zip archive
4. Create .tar.gz archive
5. Validate archive integrity

Output: helix_vpn_mvp2.zip, helix_vpn_mvp2.tar.gz